GPU Architecture

Sparsh Mittal

GPU Architecture: Thread Organization and Memory Address Spaces

We print GPU properties using following CUDA function

```
void printDevProp(cudaDeviceProp devProp) {
    printf("Major revision number:         %d\n",  devProp.major);
    printf("Minor revision number:         %d\n",  devProp.minor);
    printf("Name:                          %s\n",  devProp.name);
    printf("Total global memory:           %u\n",  devProp.totalGlobalMem);
    printf("Total shared memory per block: %u\n",  devProp.sharedMemPerBlock);
    printf("Total registers per block:     %d\n",  devProp.regsPerBlock);
    printf("Warp size:                     %d\n",  devProp.warpSize);
    printf("Maximum memory pitch:          %u\n",  devProp.memPitch);
    printf("Maximum threads per block:     %d\n",  devProp.maxThreadsPerBlock);
    for (int i = 0; i < 3; ++i)
    printf("Maximum dimension %d of block:  %d\n", i, devProp.maxThreadsDim[i]);
    for (int i = 0; i < 3; ++i)
    printf("Maximum dimension %d of grid:   %d\n", i, devProp.maxGridSize[i]);
    printf("Clock rate:                    %d\n",  devProp.clockRate);
    printf("Total constant memory:         %u\n",  devProp.totalConstMem);
    printf("Texture alignment:             %u\n",  devProp.textureAlignment);
    printf("Concurrent copy and execution: %s\n",  (devProp.deviceOverlap ? "Yes" : "No"));
    printf("Number of multiprocessors:     %d\n",  devProp.multiProcessorCount);
    printf("Kernel execution timeout:      %s\n",  (devProp.kernelExecTimeoutEnabled ? "Yes" : "No")); }
```

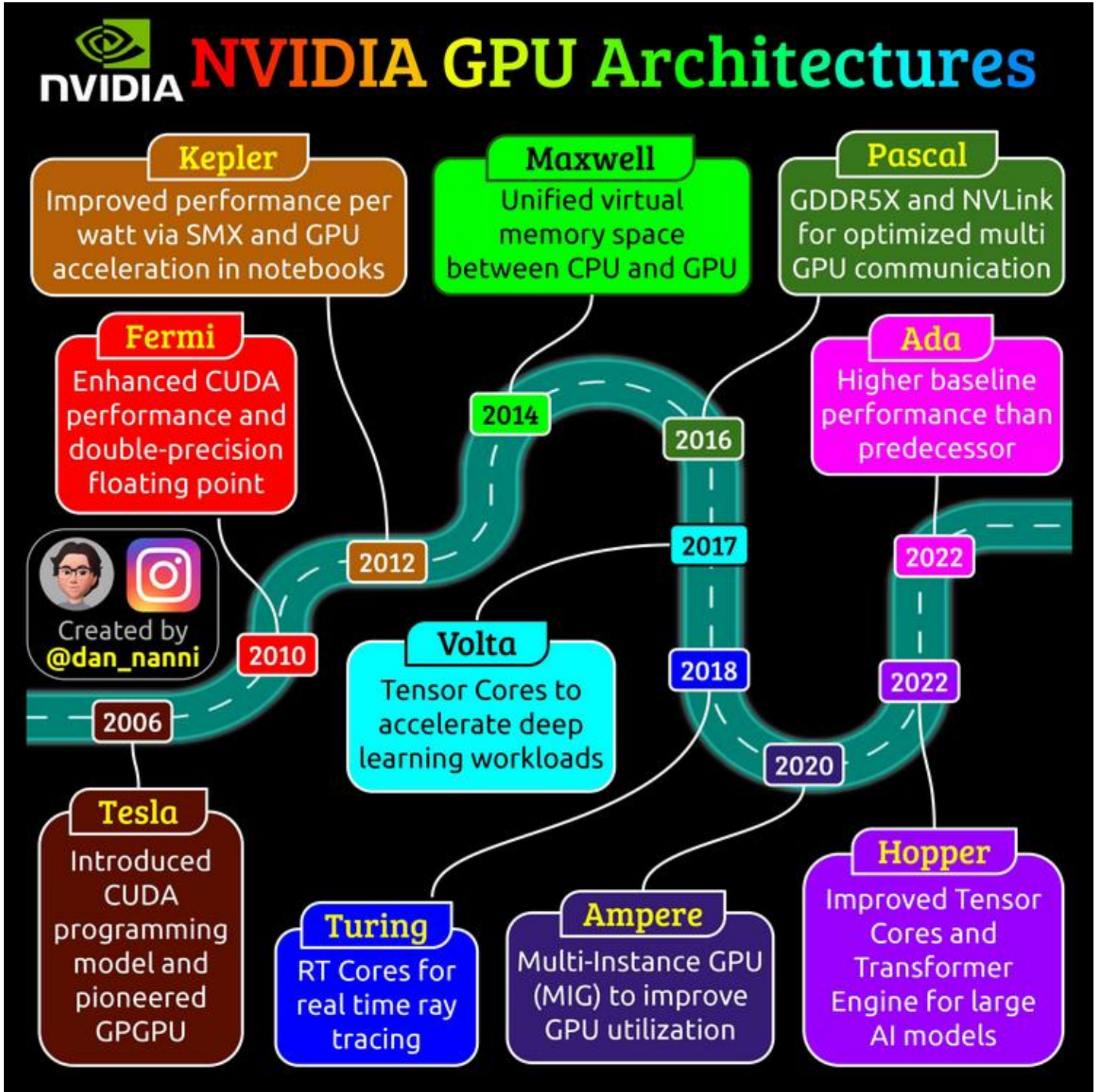
https://www.cs.fsu.edu/~xyuan/cda5125/examples/lect24/devicequery.cu

Output

```
There are 1 CUDA devices.
CUDA Device #0
Major revision number:         6
Minor revision number:         0
Name:                          Tesla P100-PCIE-12GB
Total global memory:           4200988672
Total shared memory per block: 49152
Total registers per block:     65536
Warp size:                     32
Maximum memory pitch:          2147483647
Maximum threads per block:     1024
Maximum dimension 0 of block:  1024
Maximum dimension 1 of block:  1024
Maximum dimension 2 of block:  64

Maximum dimension 0 of grid:   2147483647
Maximum dimension 1 of grid:   65535
Maximum dimension 2 of grid:   65535
Clock rate:                    1328500
Total constant memory:         65536
Texture alignment:             512
Concurrent copy and execution: Yes
Number of multiprocessors:     56
Kernel execution timeout:      No
```

	"Fermi"	"Fermi"	"Kepler"	"Kepler"	"Maxwell"	"Pascal"	"Volta"	"Turing"	"Ampere"
Tesla GPU	**GF100**	**GF104**	**GK104**	**GK110**	**GM200**	**GP100**	**GV100**	**TU104**	**GA100**
Compute Capability	2.0	2.1	3.0	3.5	5.3	6.0	7.0	7.0	8.0
Streaming Multiprocessors (SMs)	16	16	8	15	24	56	84	72	128
FP32 CUDA Cores / SM	32	32	192	192	128	64	64	64	64
FP32 CUDA Cores	512	512	1,536	2,880	3,072	3,584	5,376	4,608	8,192
FP64 Units	-	-	512	960	96	1,792	2,688	-	4,096
Tensor Core Units							672	576	512
Threads / Warp	32	32	32	32	32	32	32	32	32
Max Warps / SM	48	48	64	64	64	64	64	64	64
Max Threads / SM	1,536	1,536	2,048	2,048	2,048	2,048	2,048	2,048	2,048
Max Thread Blocks / SM	8	8	16	16	32	32	32	32	32
32-bit Registers / SM	32,768	32,768	65,536	65,536	65,536	65,536	65,536	65,536	65,536
Max Registers / Thread	63	63	63	255	255	255	255	255	255
Max Threads / Thread Block	1,024	1,024	1,024	1,024	1,024	1,024	1,024	1,024	1,024
Shared Memory Size Configs	16 KB	16 KB	16 KB	16 KB	96 KB	64 KB	Config	Config	Config
	48 KB	48 KB	32 KB	32 KB			Up To	Up To	Up To
			48 KB	48 KB			96 KB	96 KB	164 KB
Hyper-Q	No	No	No	Yes	Yes	Yes	Yes	Yes	Yes
Dynamic Parallelism	No	No	No	Yes	Yes	Yes	Yes	Yes	Yes
Unified Memory	No	No	No	No	No	Yes	Yes	Yes	Yes
Pre-Emption	No	No	No	No	No	Yes	Yes	Yes	Yes


NVIDIA GPU Architectures
Kepler
Improved performance per watt via SMX and GPU acceleration in notebooks
Maxwell
Unified virtual memory space between CPU and GPU
Pascal
GDDR5X and NVLink for optimized multi GPU communication
Fermi
Enhanced CUDA performance and double-precision floating point
Ada
Higher baseline performance than predecessor
2014
2016
2012
2017
2022
Created by
@dan_nanni
2010
Volta
Tensor Cores to accelerate deep learning workloads
2018
2022
2006
2020
Tesla
Introduced CUDA programming model and pioneered GPGPU
Hopper
Improved Tensor Cores and Transformer Engine for large AI models
Turing
RT Cores for real time ray tracing
Ampere
Multi-Instance GPU (MIG) to improve GPU utilization

- SM = streaming multiprocessor
- SP = streaming processor

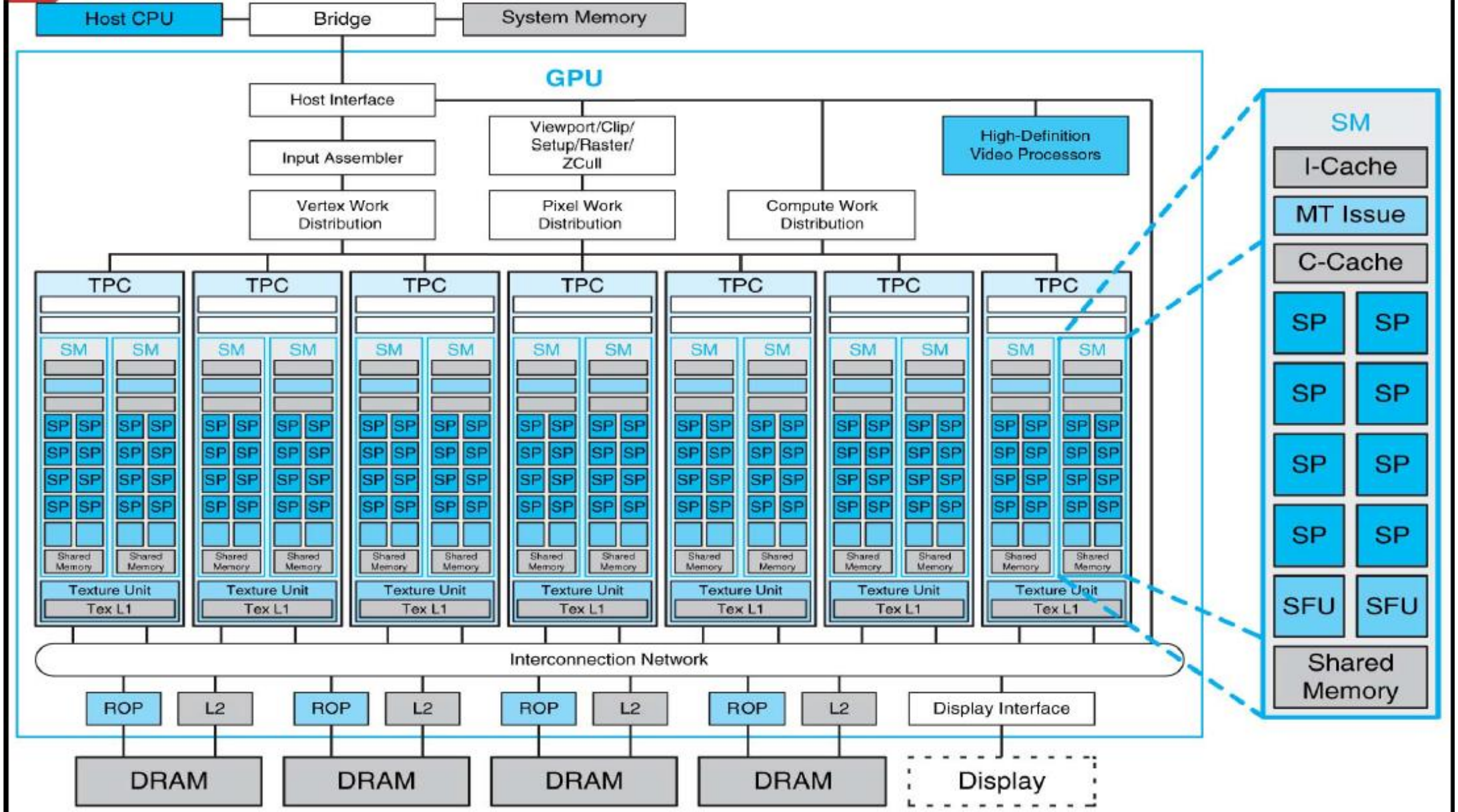


FIGURE A.2.5 Basic unified GPU architecture. Example GPU with 112 streaming processor (SP) cores organized in 14 streaming multiprocessors (SMs); the cores are highly multithreaded. It has the basic Tesla architecture of an NVIDIA GeForce 8800. The processors connect with four 64-bit-wide DRAM partitions via an interconnection network. Each SM has eight SP cores, two special function units (SFUs), instruction and constant caches, a multithreaded instruction unit and a shared memory. Copyright © 2009 Elsevier, Inc. All rights reserved.

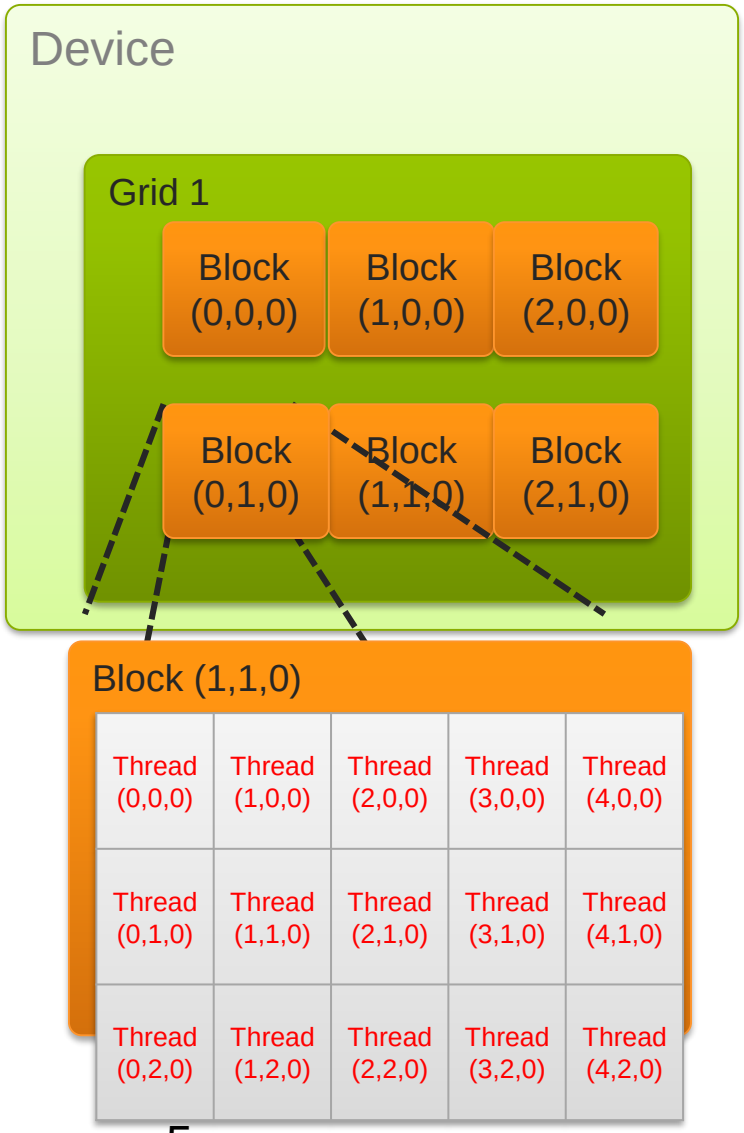
UCSB

Threads Are Organized Hierarchically

A kernel is launched as a grid of blocks of threads

Two threads from two different blocks cannot cooperate

- One thread block executes on one SM
- – All threads sharing the ‘shared memory’
- – 32 threads are executed simultaneously
- (‘warp’)



GPU Memory Address Spaces

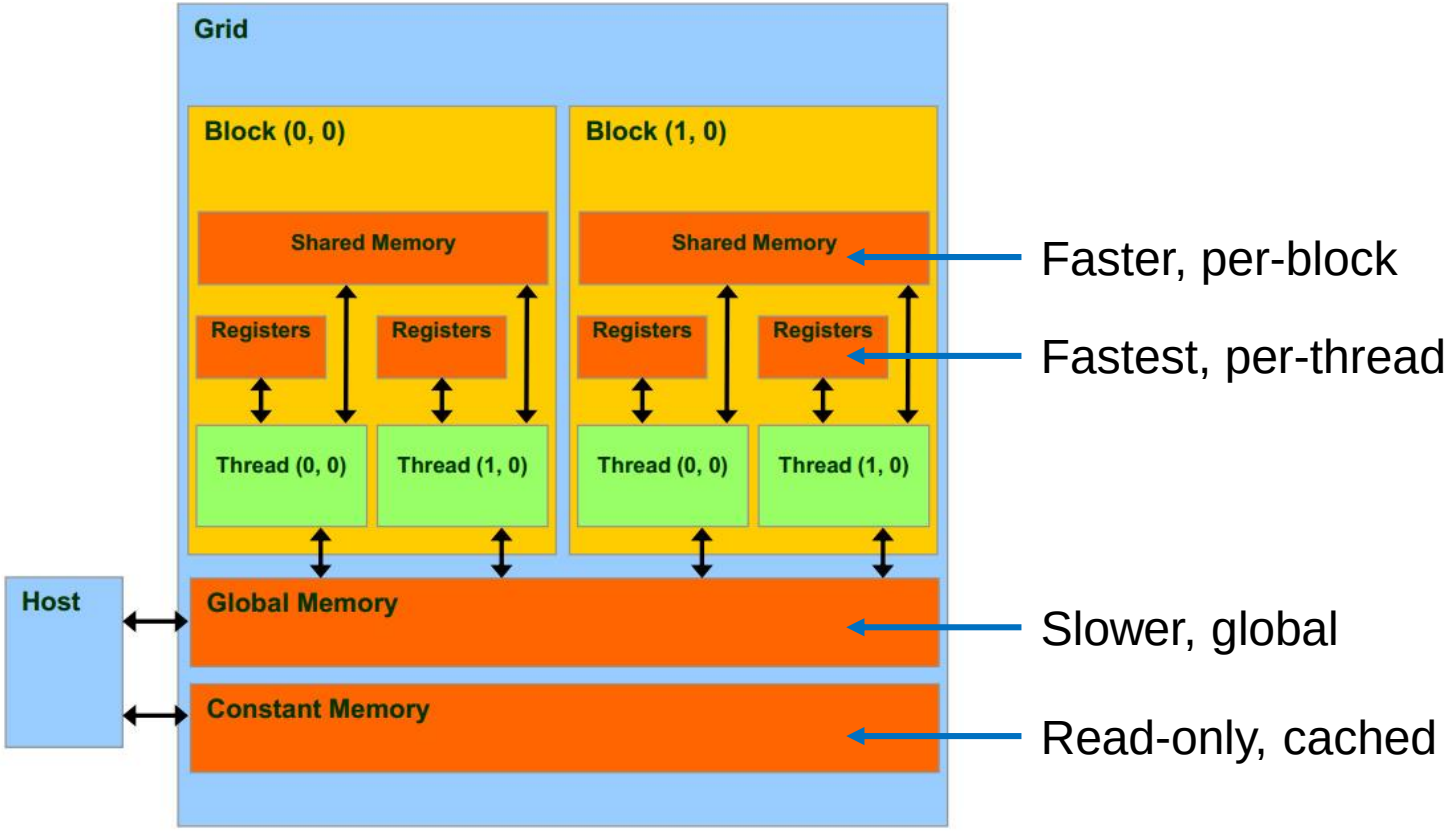
1. Local
2. Shared
3. Global

Increasing visibility of data between threads

In addition, there are two more (read-only) address spaces:

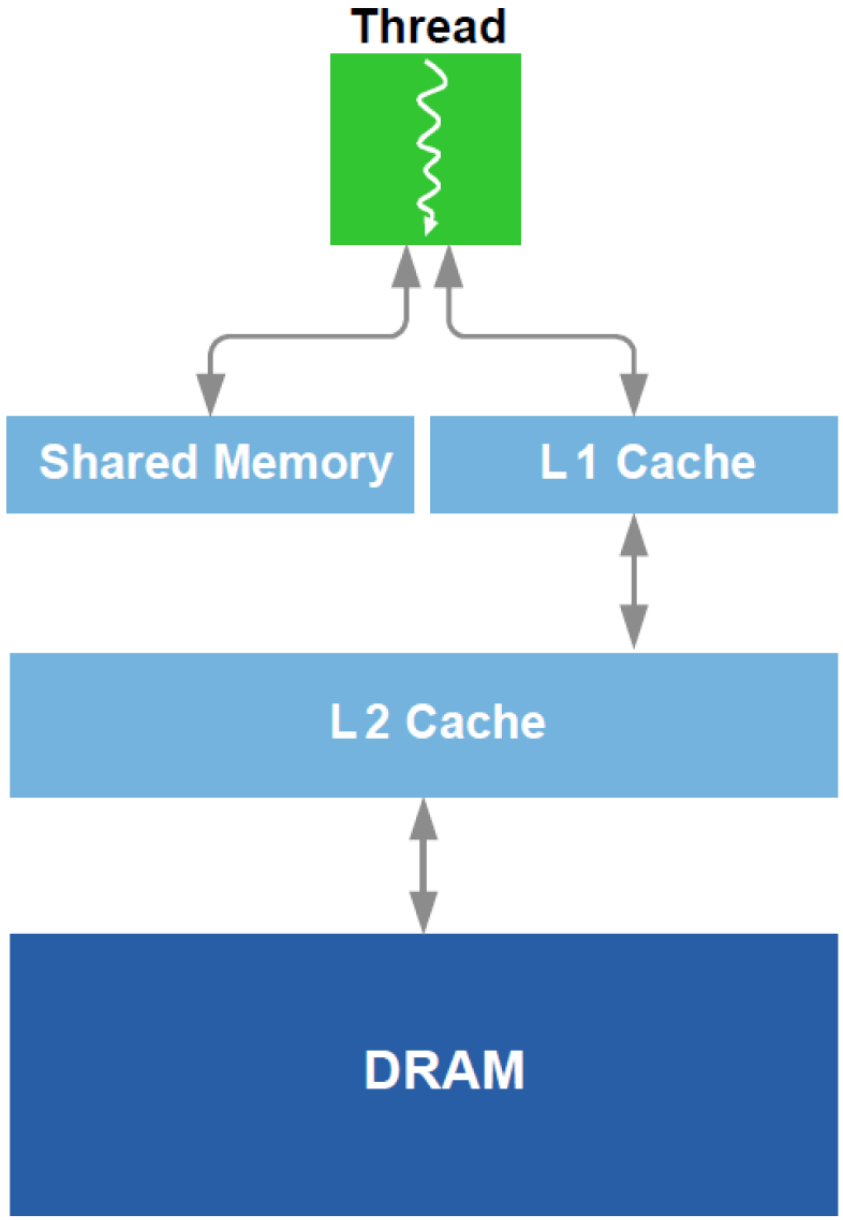
1. Constant
2. Texture

CUDA Memory

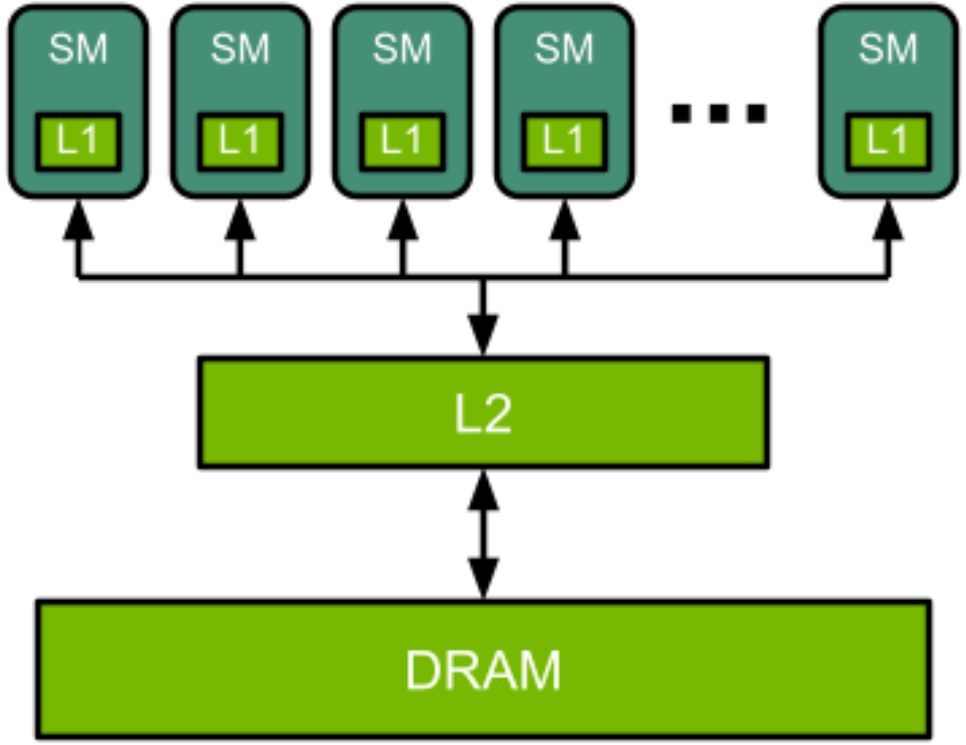

Grid
Block (0, 0)
Block (1, 0)
Shared Memory
Shared Memory
Registers
Registers
Registers
Registers
Thread (0, 0)
Thread (1, 0)
Thread (0, 0)
Thread (1, 0)
Host
Global Memory
Constant Memory
Faster, per-block
Fastest, per-thread
Slower, global
Read-only, cached

Fermi Memory Hierarchy

- L1 cache and L2 cache are hardware-managed
- Shared memory is software-managed



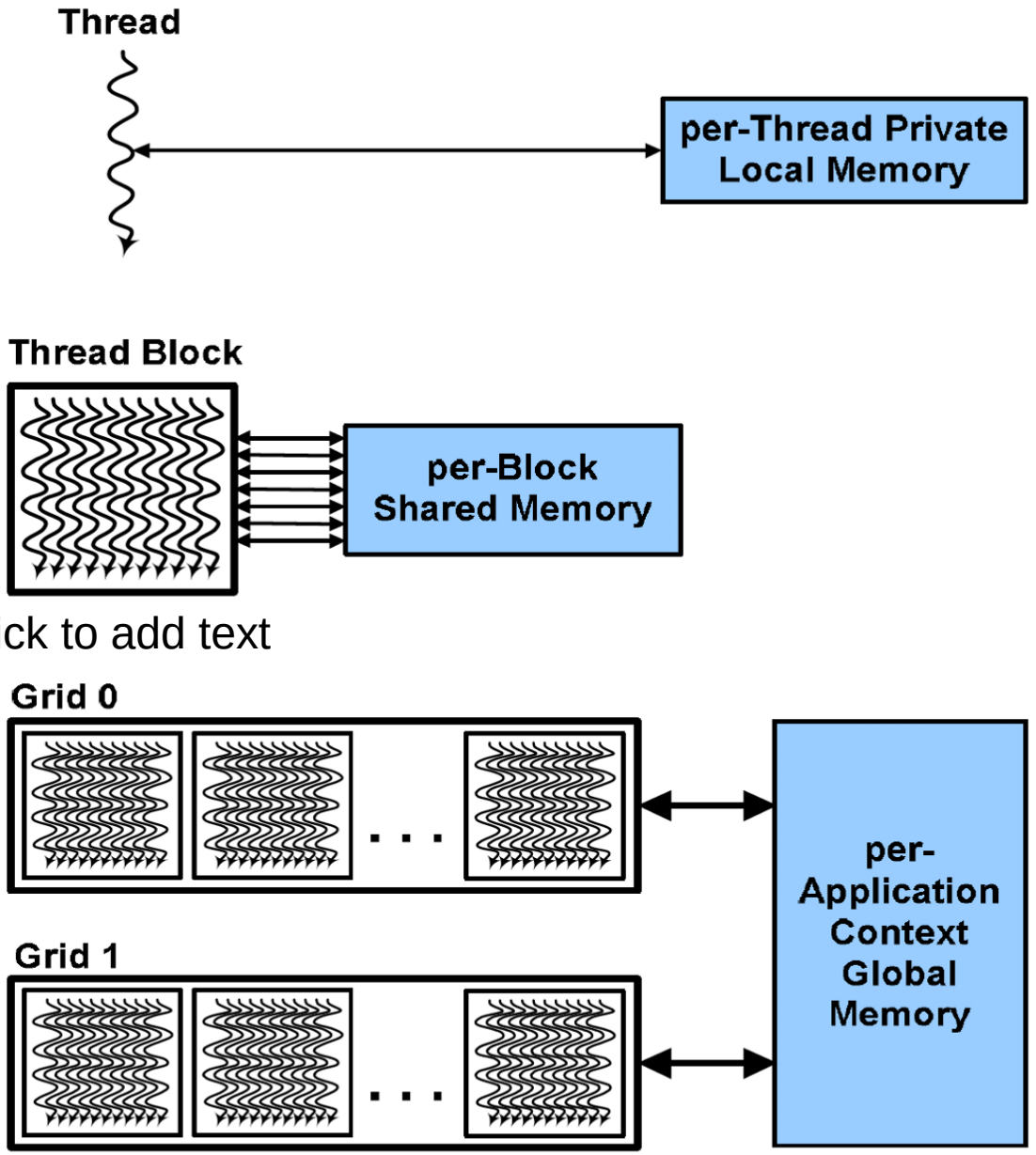
Simplified GPU architecture



GPUs consist of a number of Streaming Multiprocessors (SMs), L2 cache and DRAM.

NVIDIA A100 GPU contains 108 SMs, a 40 MB L2 cache, and up to 2039 GB/s bandwidth from 80 GB of HBM2 memory.

CUDA Hierarchy



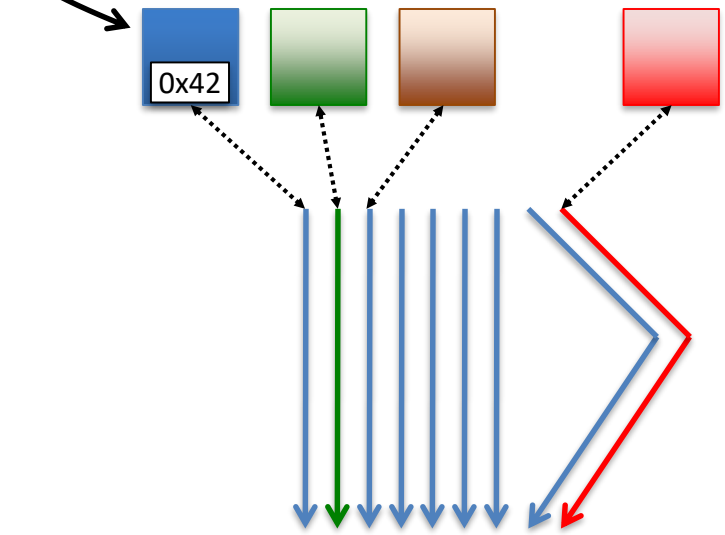
Click to add text

https://www.nvidia.com/content/PDF/fermi_white_papers/NVIDIA_Fermi_Compute_Architecture_Whitepaper.pdf

CUDA Hierarchy of threads, blocks, and grids, with corresponding per-thread private, per-block shared, and per-application global memory spaces.

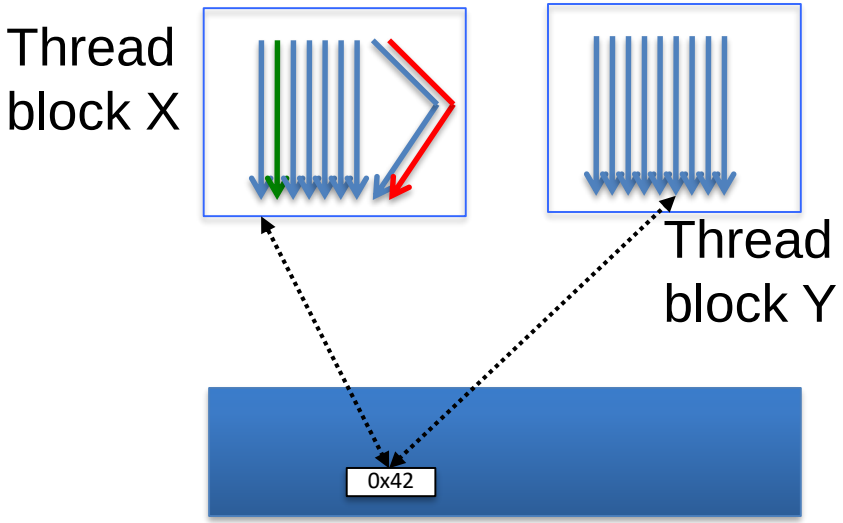
Local Memory

Each thread has its own *local memory*.



Contains local variables private to a thread.
Local memory variables are stored in DRAM, hence, access to them is slow.

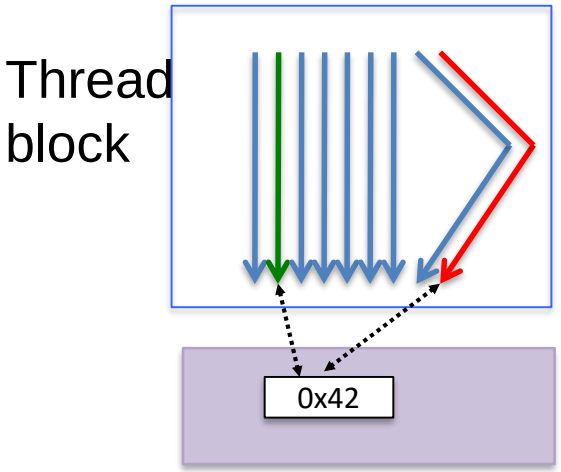
Global Memory



- Each thread in the different thread blocks (even from different kernels) can access *global memory*
- **cudaMalloc** in CUDA allocates global memory

CUDA = Compute unified device architecture

Shared Memory

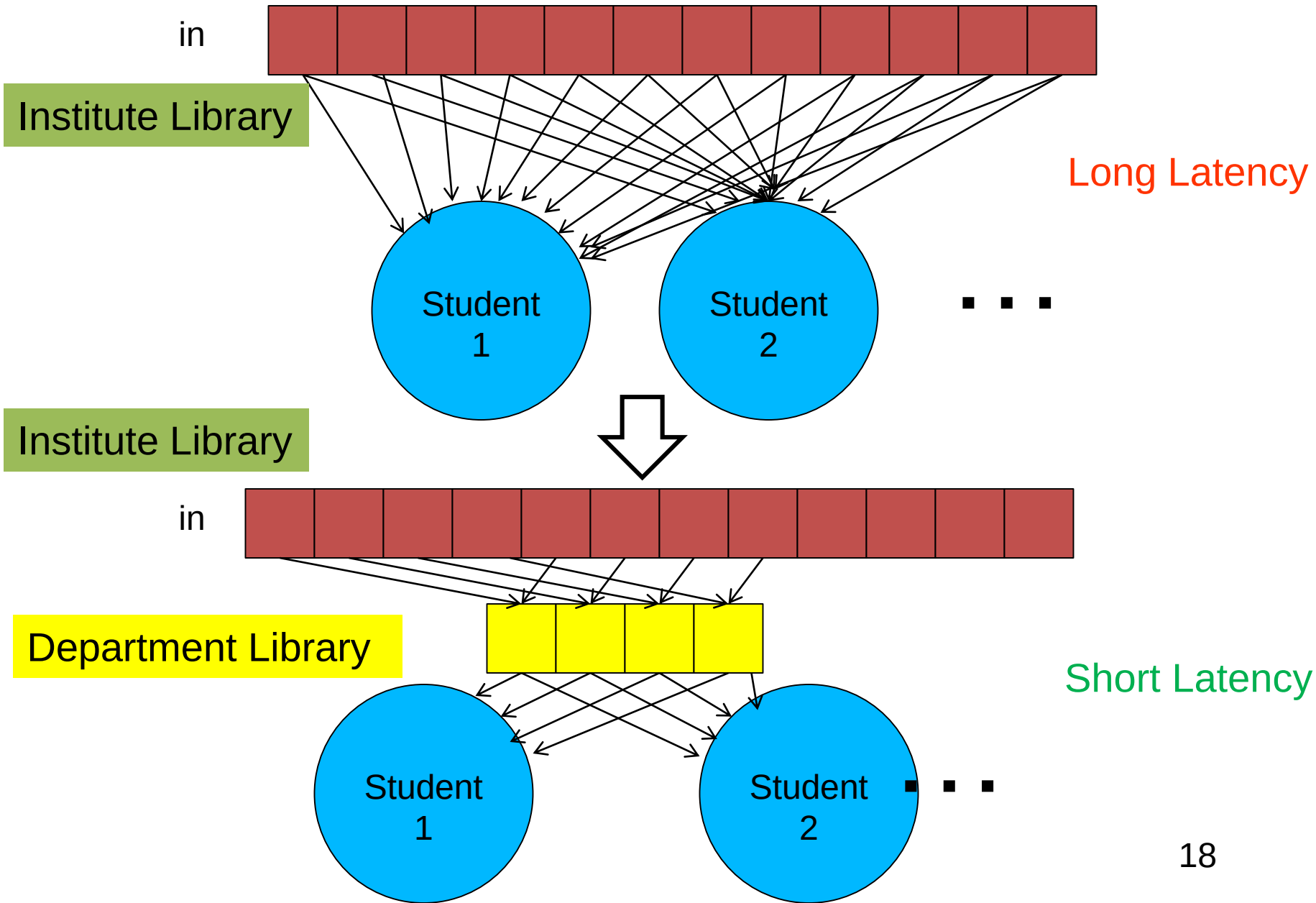


Each thread in the same block can access a memory region called *shared memory*

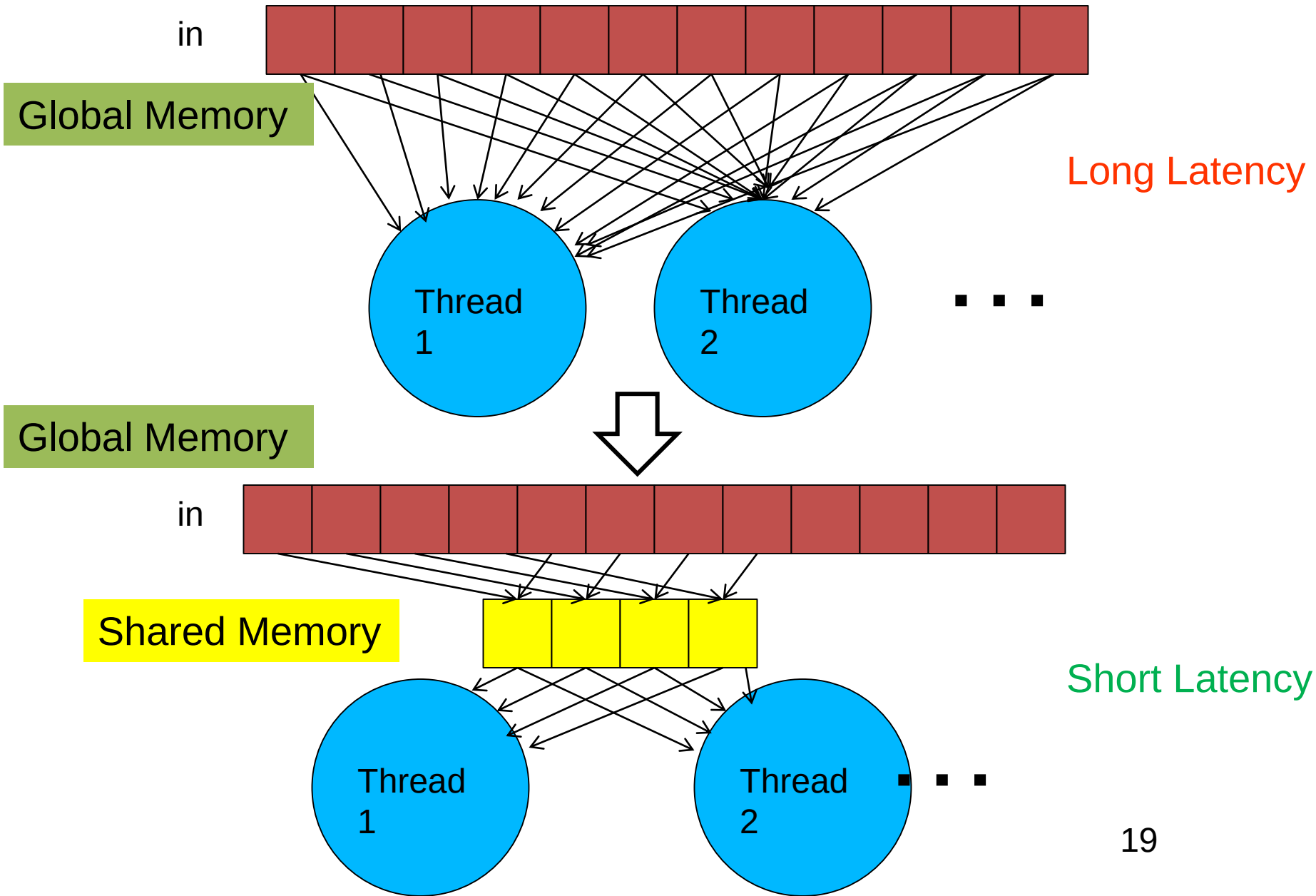
Limited size (tens of KB)

Used as a software-managed *cache* to avoid off-chip memory accesses

Analogy: Institute and Dept. Library



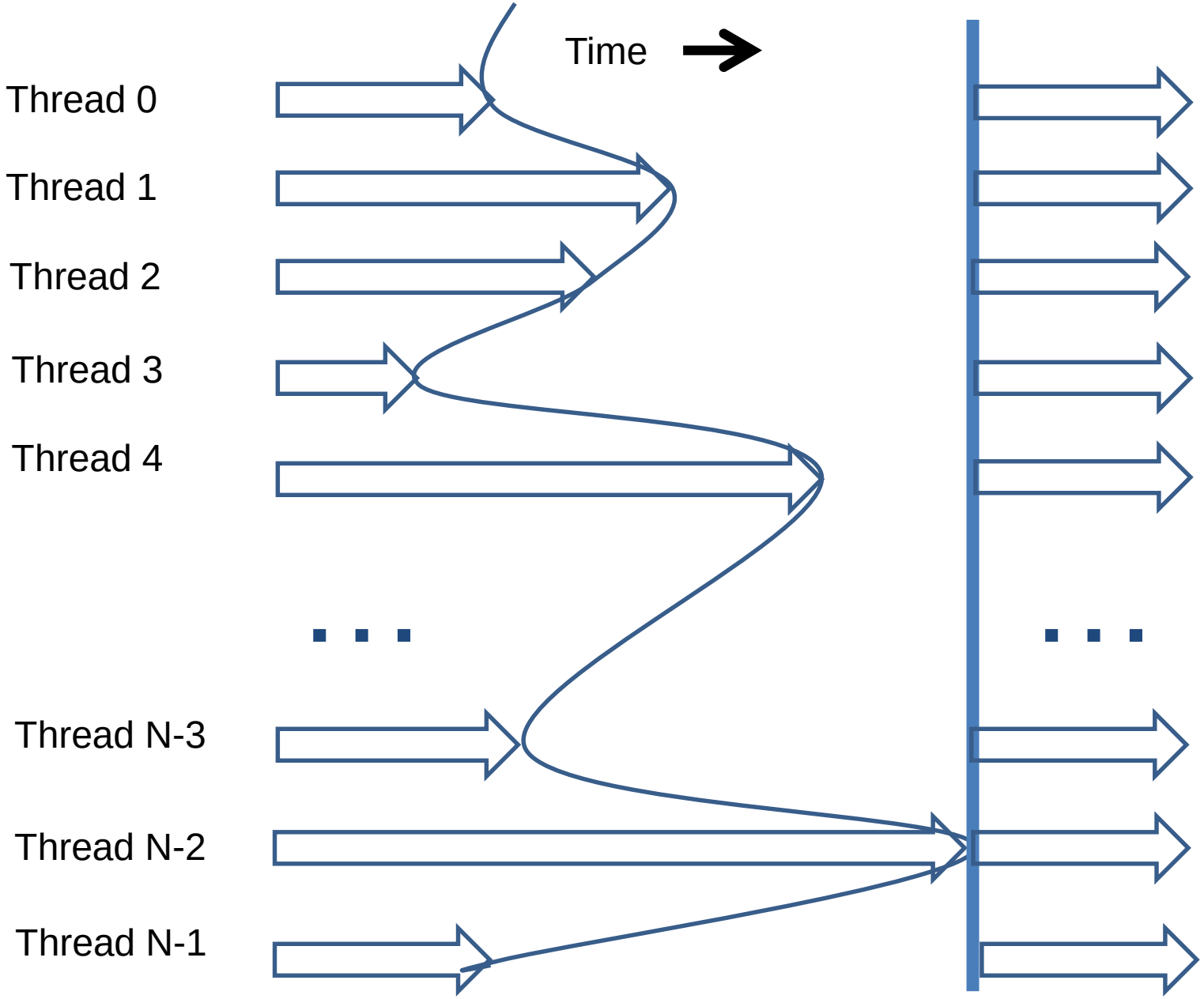
Similarly, Global and Shared Memory



Parallel Programming Is Not Easy!

Shared memory requires synchronization

https://www.reddit.com/r/aww/comments/2oagj8/multithreaded_programming_theory_and_practice/



An example of execution timing of barrier synchronization.

Barrier Synchronization

- A function call in CUDA
  - __ syncthreads()

- All threads in the same block must reach the __ syncthreads() before any can move on

CUDA Variable Type Qualifiers

Variable declaration	Memory	Scope	Lifetime	Latency
`int LocalVar;`	register	thread	thread	1x
`int localArray[10];`	local	thread	thread	100x
`__shared__ int SharedVar;`	shared	block	block	1x
`__device__ int GlobalVar;`	global	grid	application	100x
`__constant__ int ConstVar;`	constant	grid	application	1x

- Automatic variables without any qualifier reside in a register
  - Except per-thread arrays that reside in local memory
  - Or if there are not enough registers

Access Location	Access Latency (cycles)
Register	1
Shared Memory	29
L1D cache	37.9
L2 cache	261.5
Global Memory	466.3

Access Latencies for various levels of the A100 GPU memory hierarchy

Device	LLC cache size (MB)	L1 cache size (KB)
A100	40	192
H100	50	256
L40	96	128
RTX4090	72	128

Cache capacities for server-grade GPUs

References

- CUDA language:
  - CUDA by Example, by Jason Sanders and Edward Kandrot, NVIDIA
  - "Programming Massively Parallel Processors: A Hands-on Approach" by David B. Kirk and Wen-mei W. Hwu
- GPU architecture
  - "A Survey of CPU-GPU heterogeneous computing", S. Mittal et al., CSUR 2015 (pdf)
  - "A Survey of Techniques for Optimizing Deep Learning on GPUs", S. Mittal et al., JSA 2019 (pdf)
  - "A Survey on Optimized Implementation of Deep Learning Models on the NVIDIA Jetson Platform", S. Mittal, JSA, 2019 (pdf)
  - https://cvw.cac.cornell.edu/gpu/coalesced